

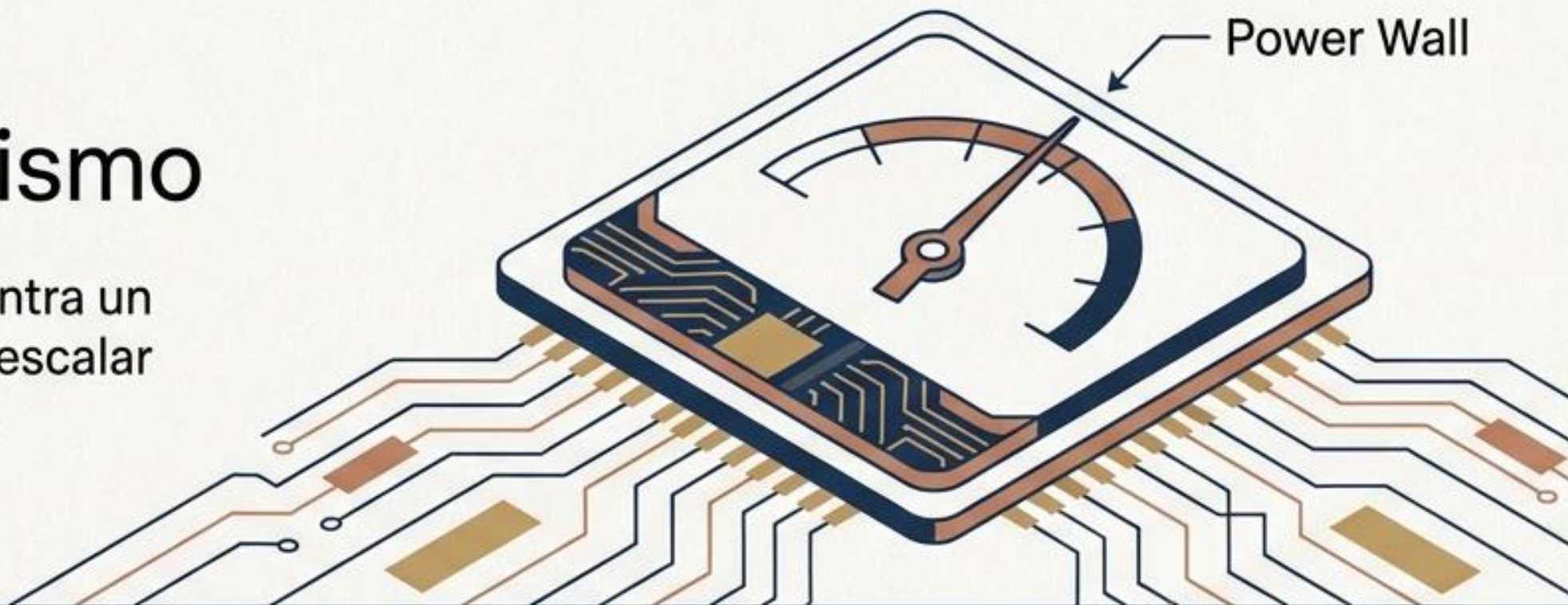


El Paradigma de la Computación Paralela

Desde los límites teóricos del silicio hasta la optimización a nivel de bits.

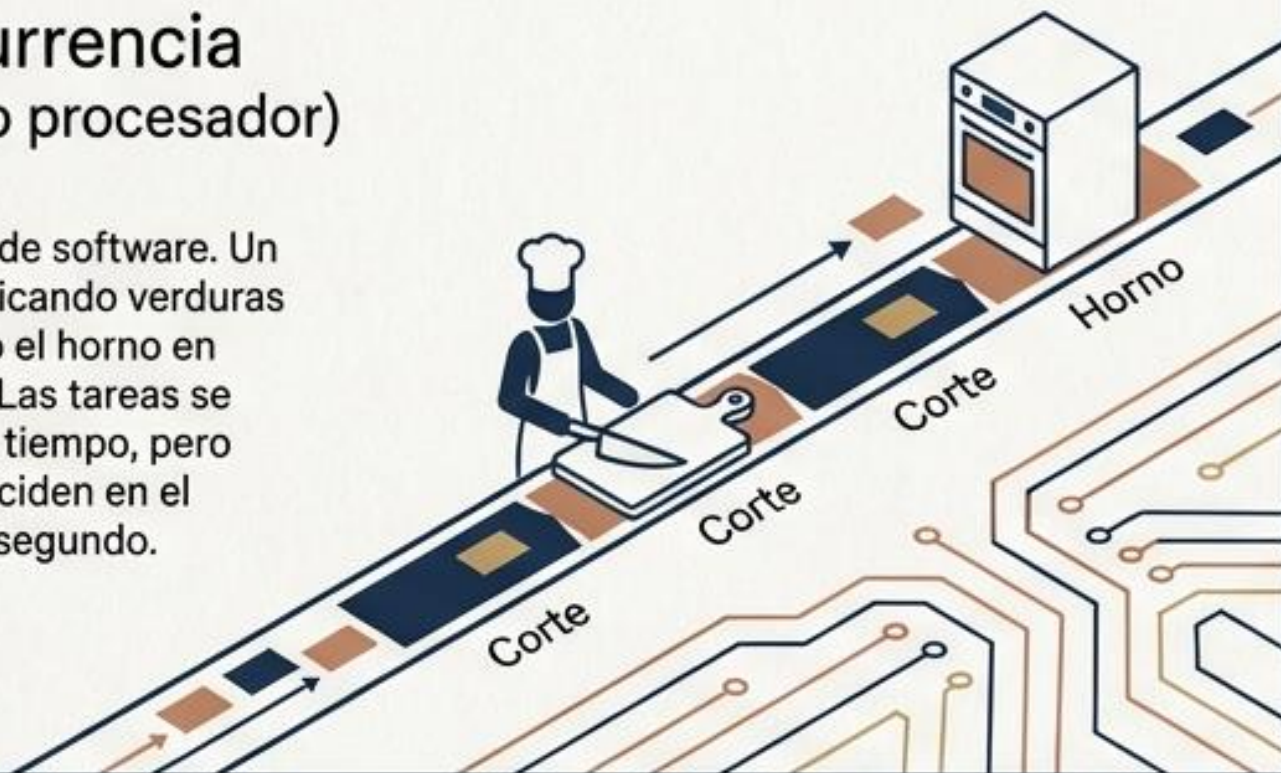
El límite térmico y la necesidad del paralelismo

La computación secuencial ha chocado contra un muro de potencia física. La única vía para escalar el rendimiento es la ejecución simultánea.



Concurrencia (Un solo procesador)

Estructura de software. Un solo chef picando verduras y revisando el horno en intervalos. Las tareas se solapan en tiempo, pero nunca coinciden en el mismo milisegundo.



Paralelismo (Múltiples procesadores)

Estructura de hardware. Dos chefs trabajando físicamente al mismo tiempo: uno pica verduras, el otro hornea. Ejecución física simultánea.



Los límites teóricos de la aceleración

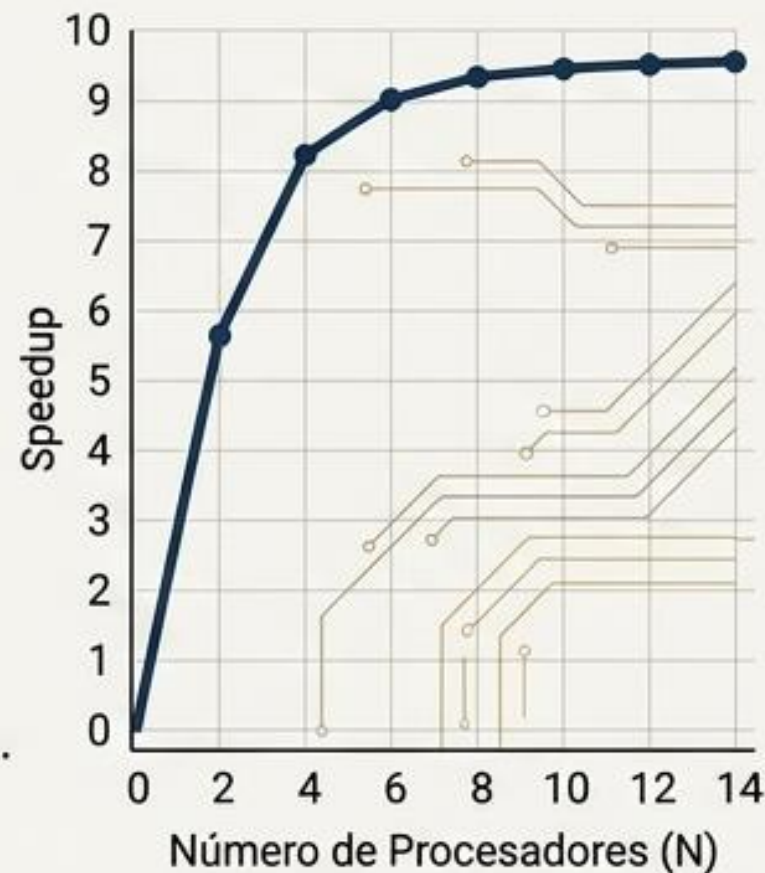
Ley de Amdahl (1967)

Visión pesimista. Tamaño del problema fijo. El rendimiento está limitado por la fracción secuencial (el cuello de botella).

$$\text{Speedup} = \frac{1}{(1 - p) + \frac{p}{N}}$$

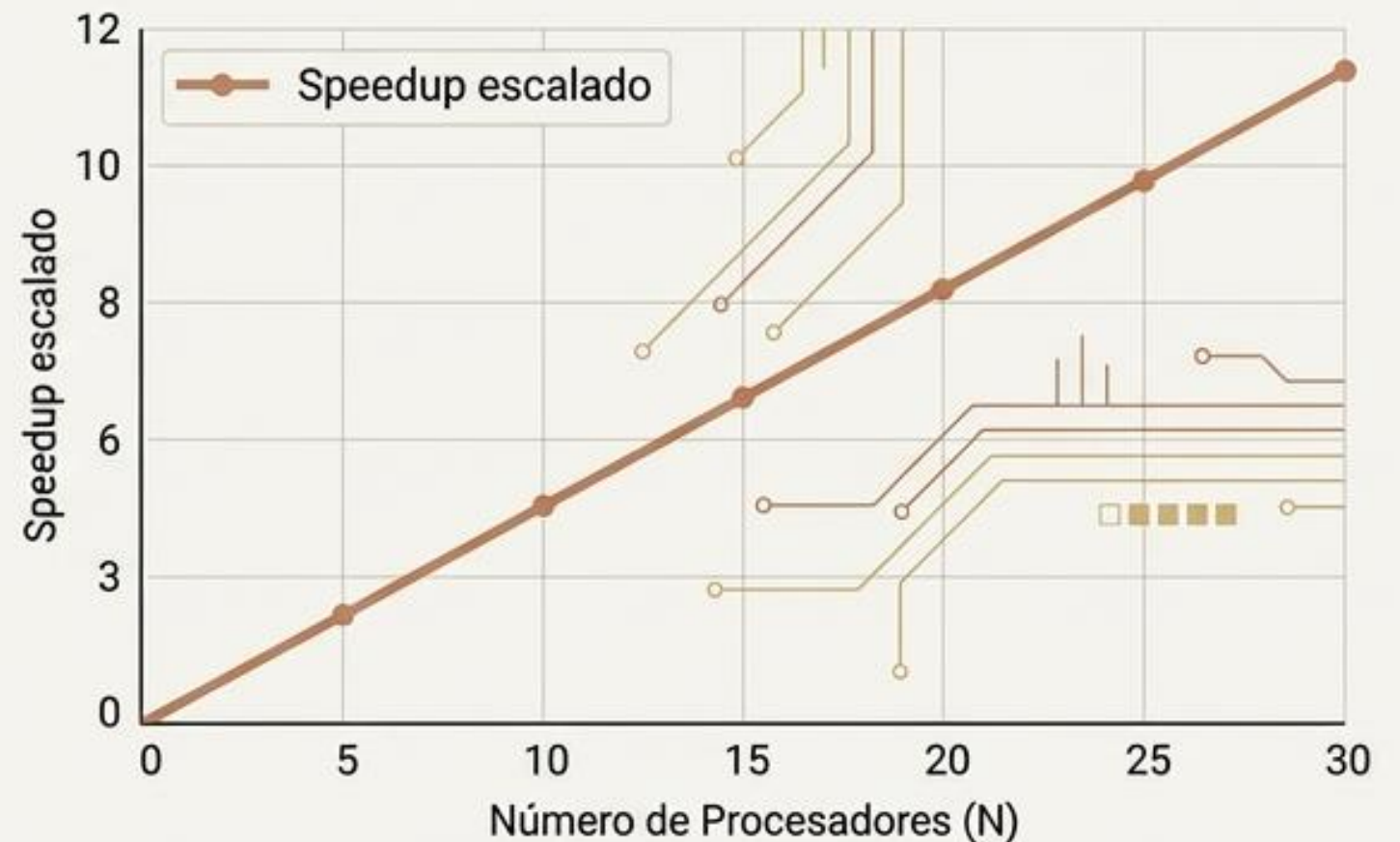
Donde la fracción paralelizable es representada por la letra p , y el número de procesadores es N .

Si el 10% del código es secuencial, la aceleración máxima nunca superará 10x.

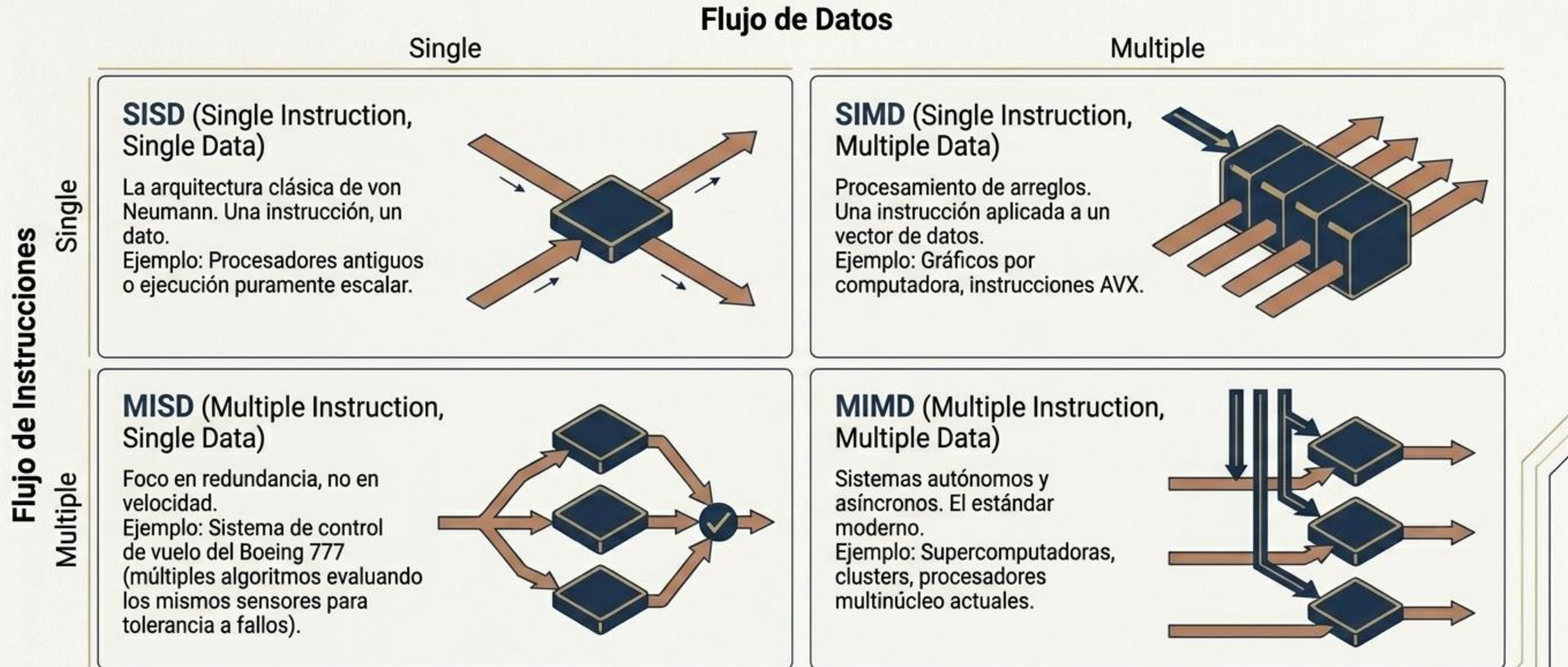


Ley de Gustafson-Barsis (1988)

Visión optimista. Tamaño del problema escalable. Al añadir procesadores, los científicos no resuelven el mismo problema más rápido; resuelven problemas mucho más masivos en el mismo tiempo. Crecimiento lineal.



Taxonomía de Flynn: La matriz arquitectónica



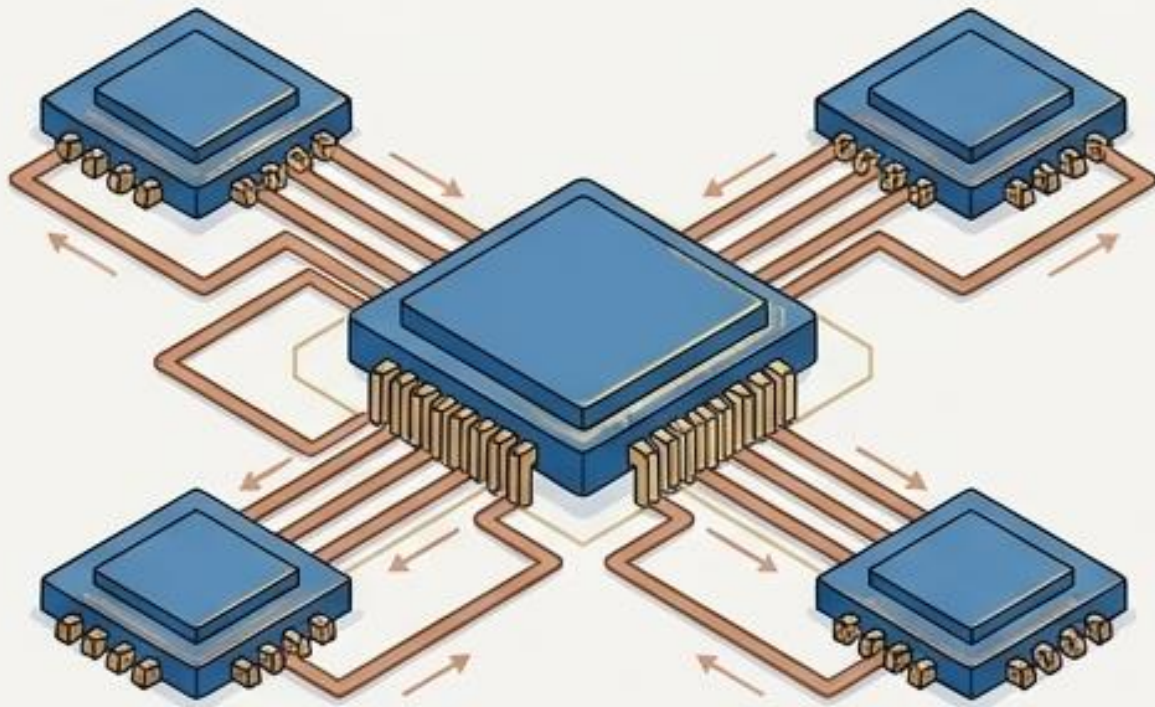
La jerarquía de memoria en sistemas MIMD

Memoria Compartida (UMA/NUMA)

Múltiples núcleos acceden a un espacio de direcciones global.

Pros/Contras

- Fácil de programar, pero difícil de escalar.
- Riesgo de Falsa Compartición y cuellos de botella en el bus de sistema.
- Requiere protocolos de coherencia de caché (MESI/MOESI).

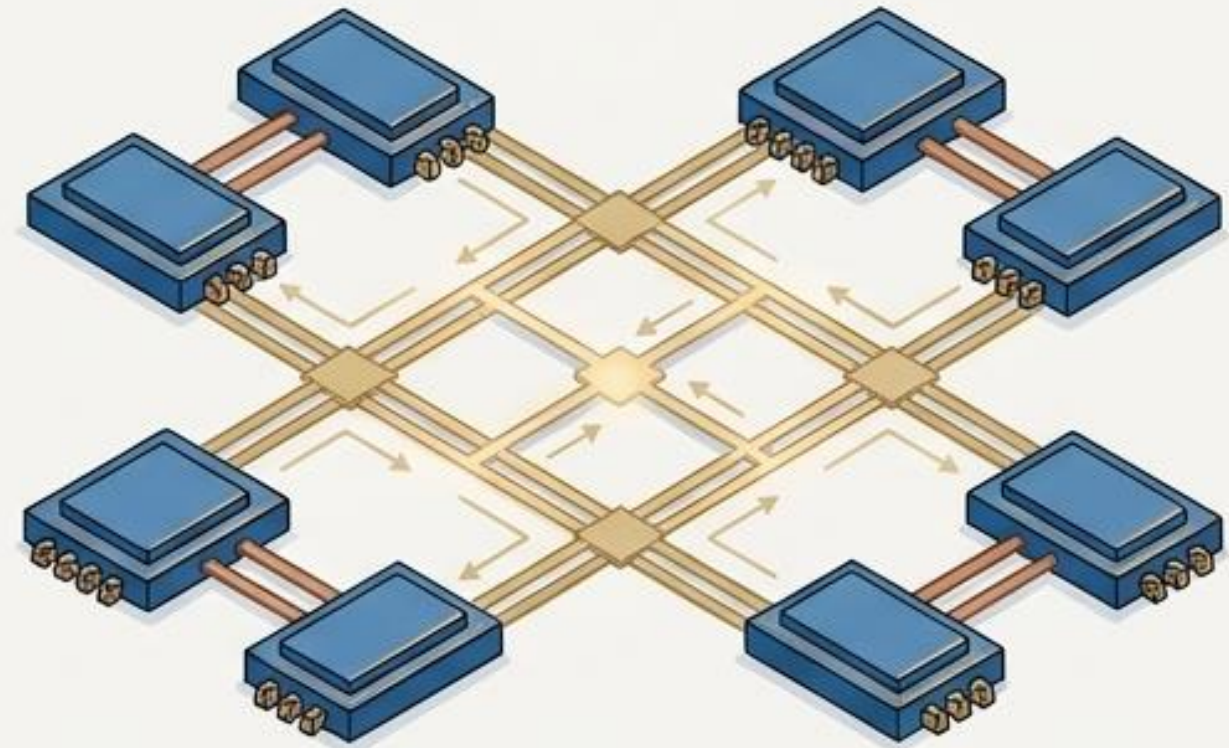


Memoria Distribuida (Paso de Mensajes)

Cada nodo posee su memoria privada (Local RAM); los datos se comparten a través de la red (ej. estándar MPI).

Pros/Contras

- Altamente escalable (base de clusters top500), pero mayor latencia y mayor complejidad de programación.

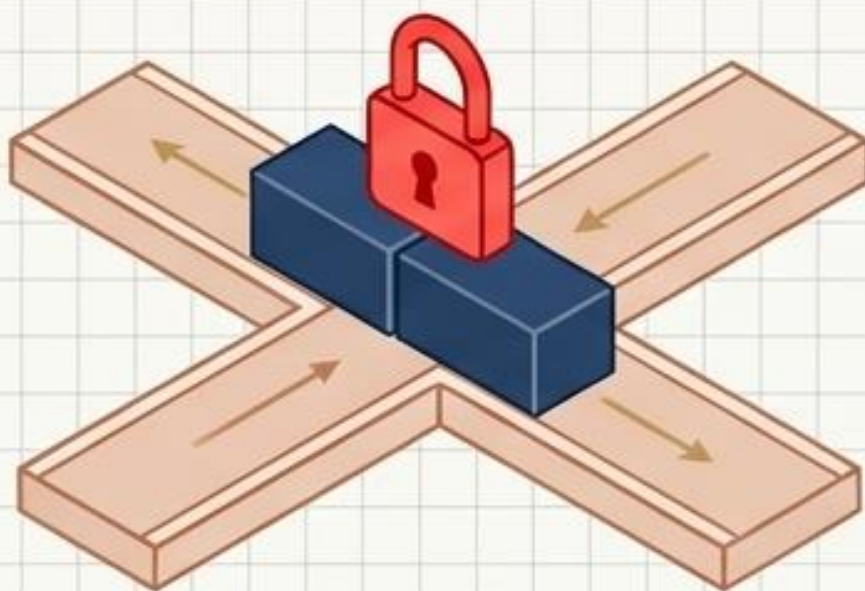


Riesgos de sincronización y estados de bloqueo

Sin coordinación (semáforos, mutex), el acceso paralelo corrompe los datos o detiene el sistema.

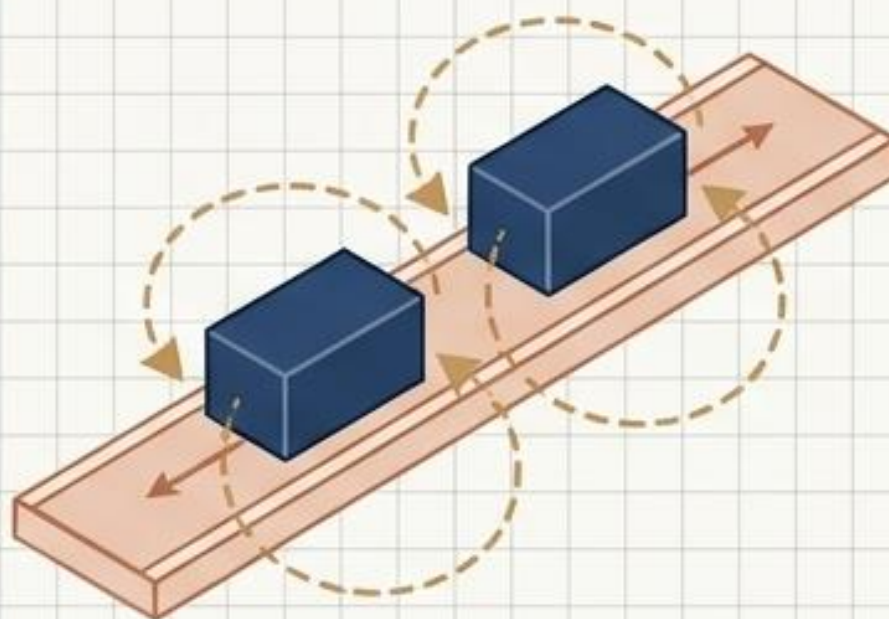
Deadlock (Interbloqueo)

Dos hilos en un pasillo estrecho. Ambos se detienen esperando que el otro retroceda. El sistema se congela permanentemente (Condiciones de Coffman).



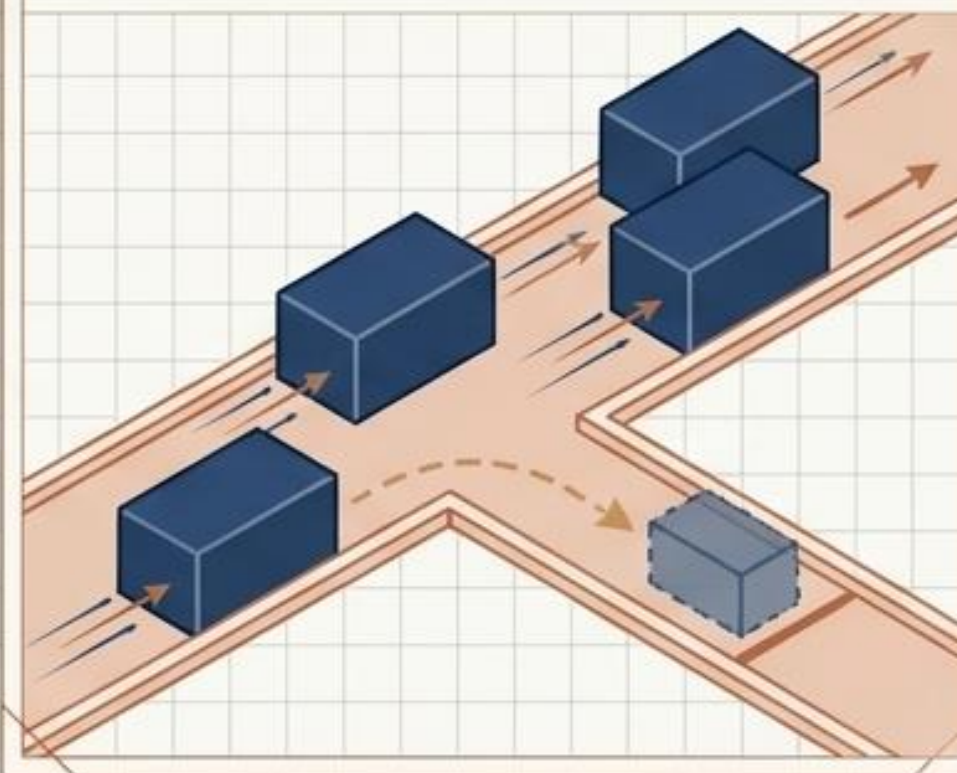
Livelock

Los hilos cambian activamente de estado intentando esquivarse, pero oscilan en el mismo ciclo temporal. Hay actividad, pero cero progreso.



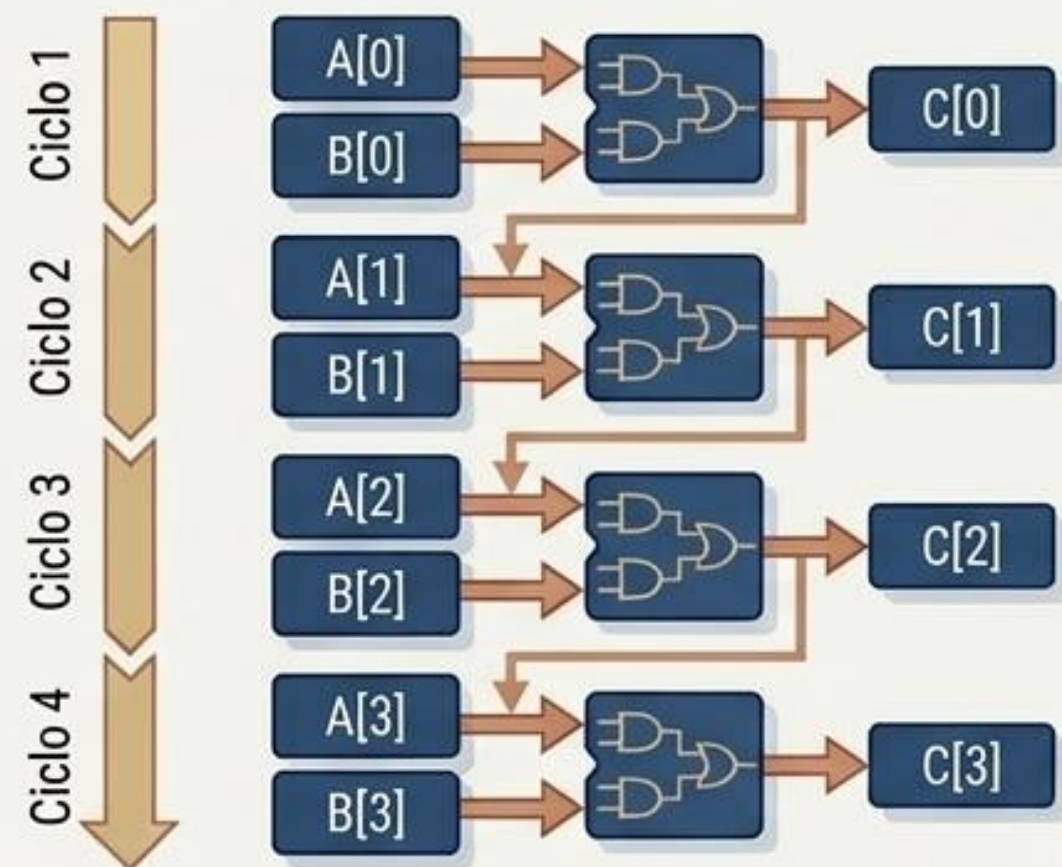
Starvation (Inanición)

Un hilo nunca obtiene acceso a los recursos porque otros hilos de mayor prioridad acaparan el sistema continuamente.



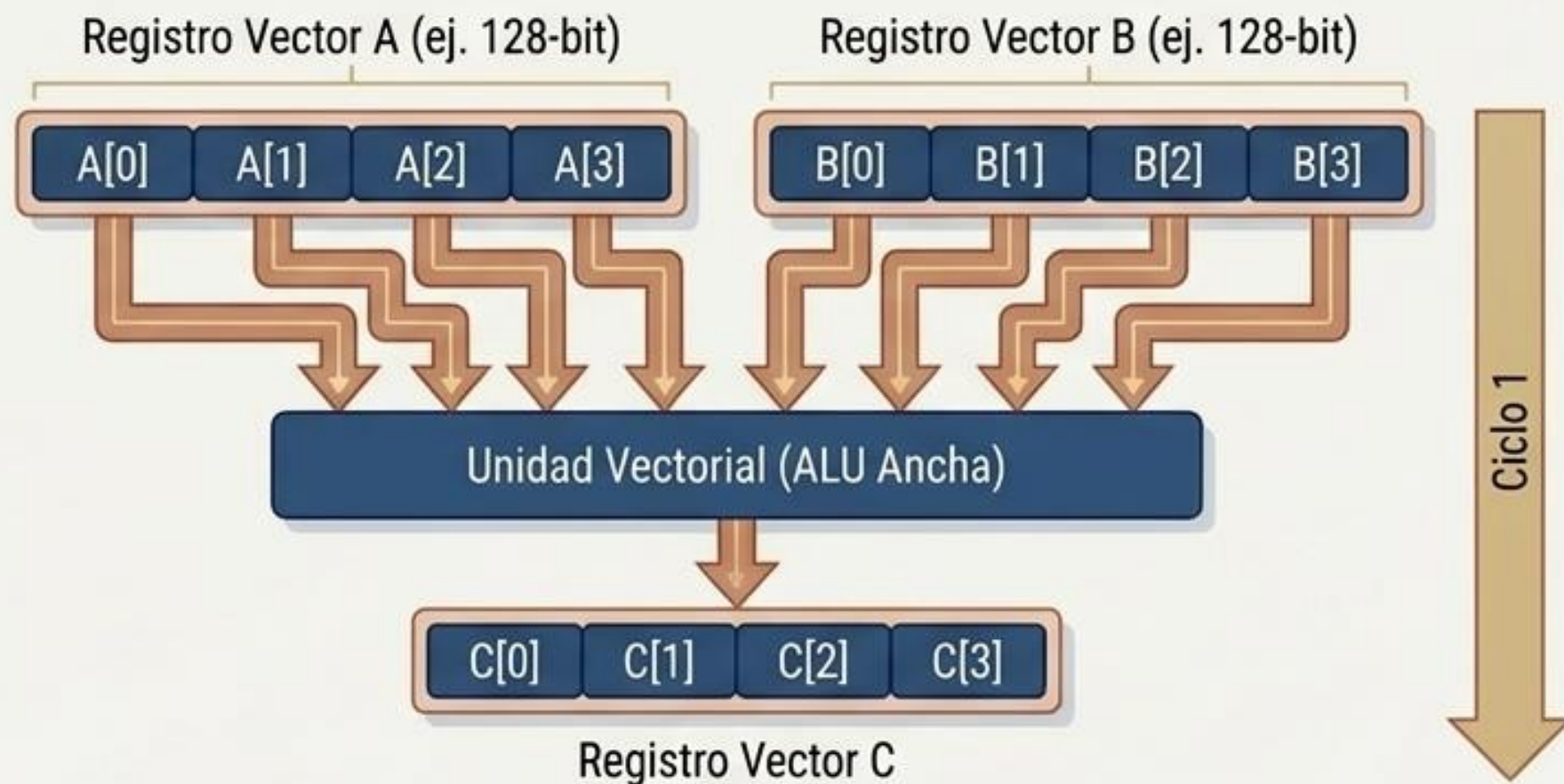
La evolución SIMD: Paralelismo a nivel de datos

Ejecución Escalar (El problema)



Toma **4 ciclos de reloj**
sumar 4 pares de números.

Ejecución Vectorial (La solución SIMD)



Toma **1 ciclo de reloj**
procesar la cadena completa.

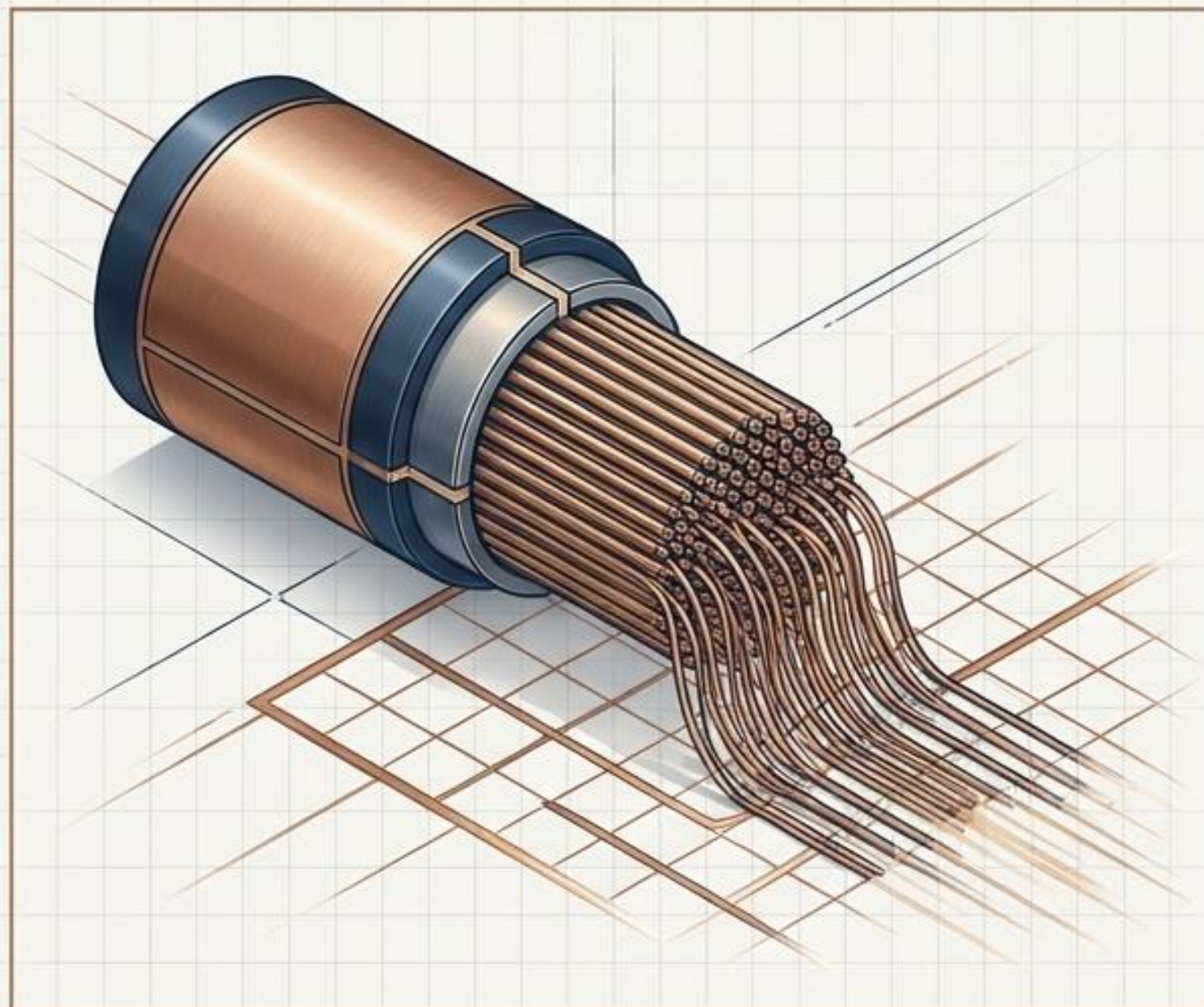
Key Insight: El hardware SIMD agrupa elementos de datos en vectores de longitud fija (ej. 32-bit a 128-bit) aplicando una única instrucción, ideal para procesamiento de imágenes y matrices.

La Revolución SIMT: El modelo de las GPUs

Single Instruction, Multiple Threads. Nvidia introdujo una abstracción que revolucionó el hardware gráfico hacia la computación general.

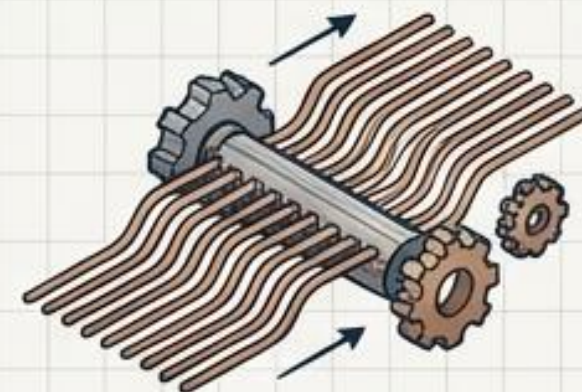
Concepto 1: El Hilo como abstracción

En lugar de obligar al programador a empacar datos manualmente en vectores rígidos, SIMT permite escribir código como si fuera para un solo hilo.



Concepto 2: El Warp

El hardware agrupa los hilos en bloques de 32 (Warps). Estos 32 hilos se ejecutan en estricto lock-step (al unísono) en los núcleos CUDA. Si uno avanza, todos avanzan.



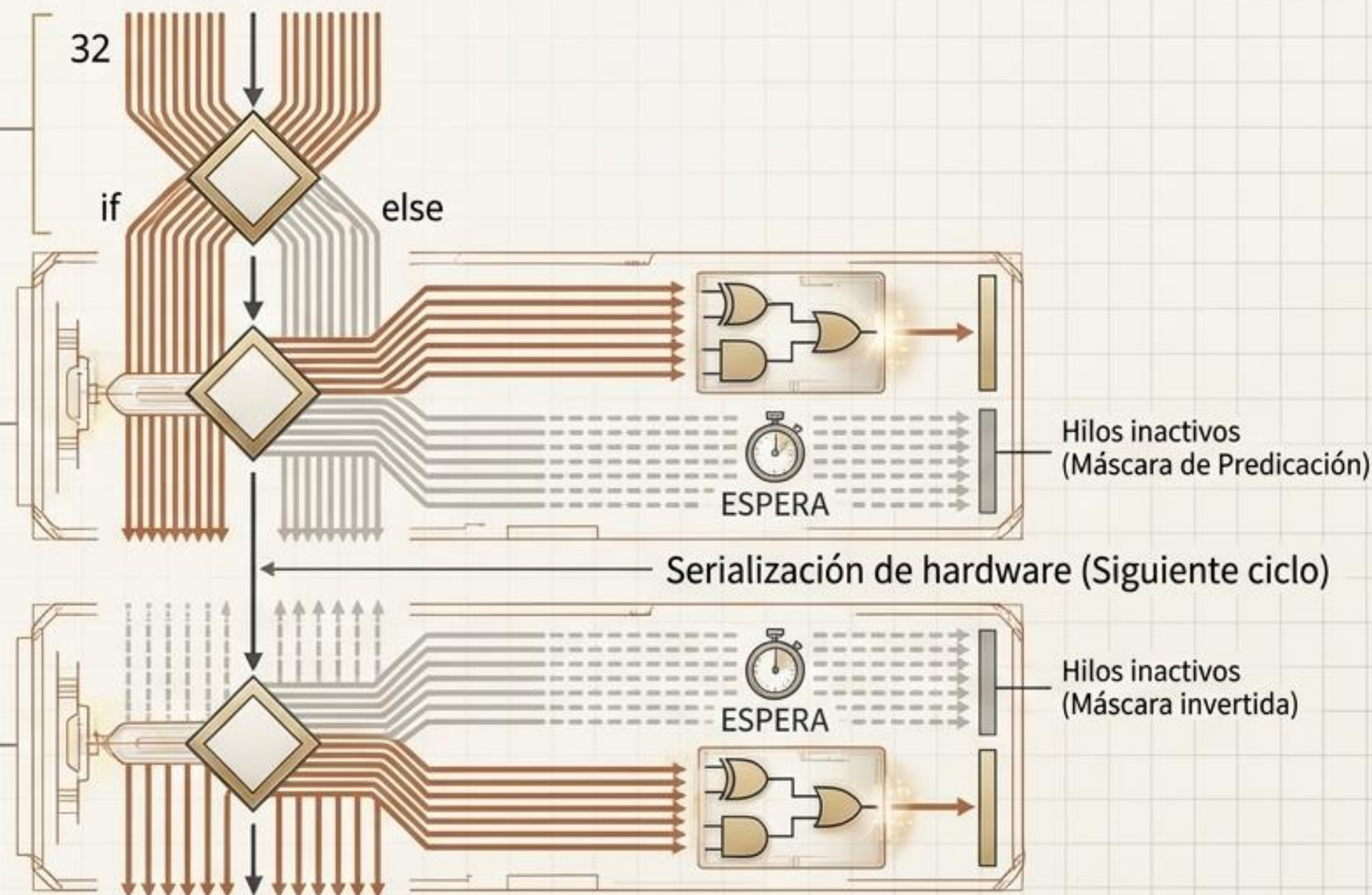
El costo de la divergencia en SIMT

¿Qué ocurre si un Warp encuentra una instrucción if/else y los hilos toman caminos distintos?

Divergencia: El Warp llega a la condición. 16 hilos cumplen el if, 16 hilos van al else.

Máscara de Predicación - Camino A: El hardware serializa la ejecución. Primero, ejecuta el código if. Los hilos inactivos esperan sin hacer trabajo útil.

Máscara de Predicación - Camino B: Luego, se invierte la máscara y se ejecuta el código else.



Key Insight: La divergencia destruye el rendimiento. El código GPU perfecto evita ramas condicionales dentro del mismo Warp.

La pila de programación paralela

Nivel 4: Algoritmos y Patrones

Diseño de control (Map, Stencil, Reduction, Scan, Master/Slave).

Nivel 3: Lenguajes y Compiladores

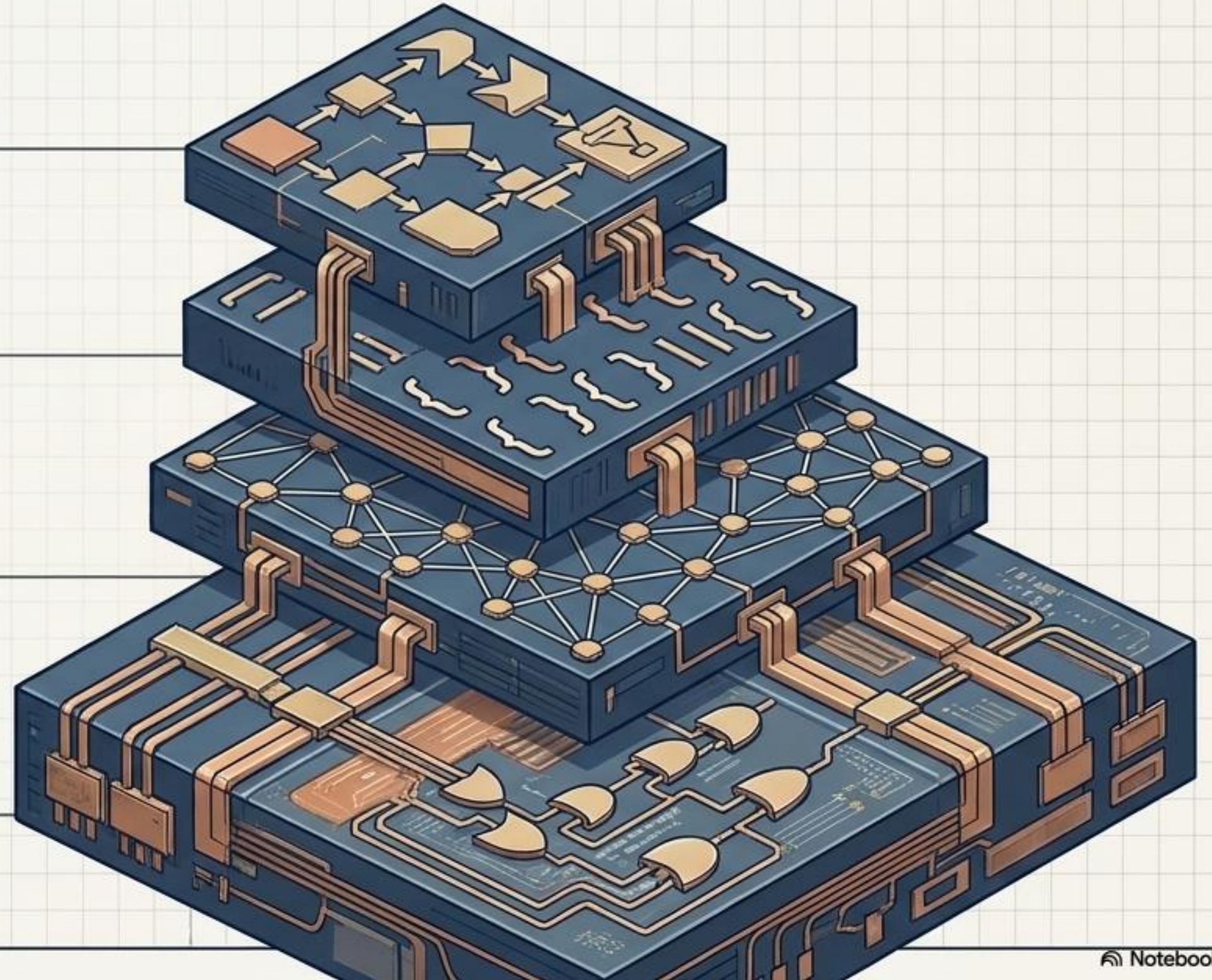
C++, Rust, Fortran. Detección de paralelismo manual vs automático.

Nivel 2: Interfaces y APIs (El puente)

OpenMP (Memoria compartida, directivas).
MPI (Memoria distribuida).
CUDA / OpenCL (Programación heterogénea).

Nivel 1: Hardware y Registros

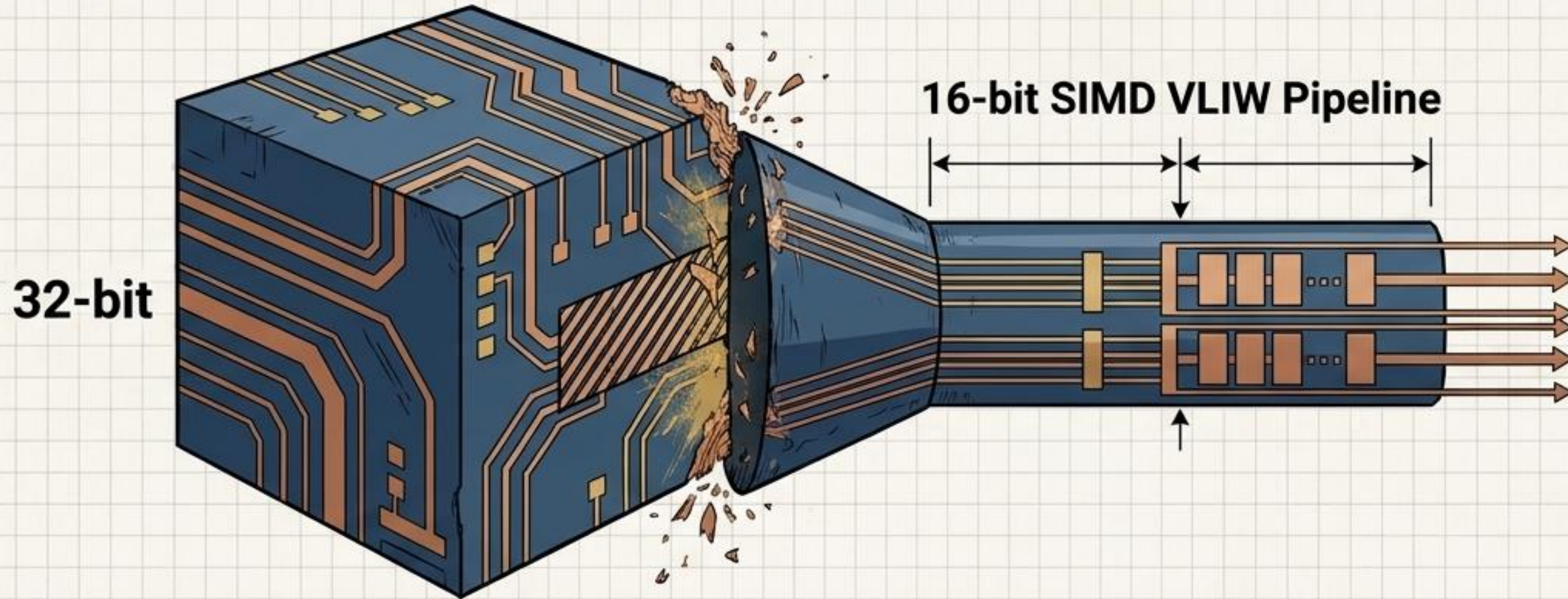
CPU, GPU, DSP, FPGAs. La memoria física y el diseño de la ALU. Aquí es donde ocurren las verdaderas restricciones de ingeniería.



Caso de Estudio: Multiplicación matricial al límite del silicio

The Hardware Reality

A veces es necesario implementar algoritmos modernos (multiplicación de matrices de 32 bits con coma fija) en plataformas embebidas u obsoletas.



The Constraint

Un DSP de bajo consumo (400 MHz) sin soporte para punto flotante. Posee un pipeline VLIW y registros vectoriales SIMD limitados estrictamente a 16 bits.

The Challenge

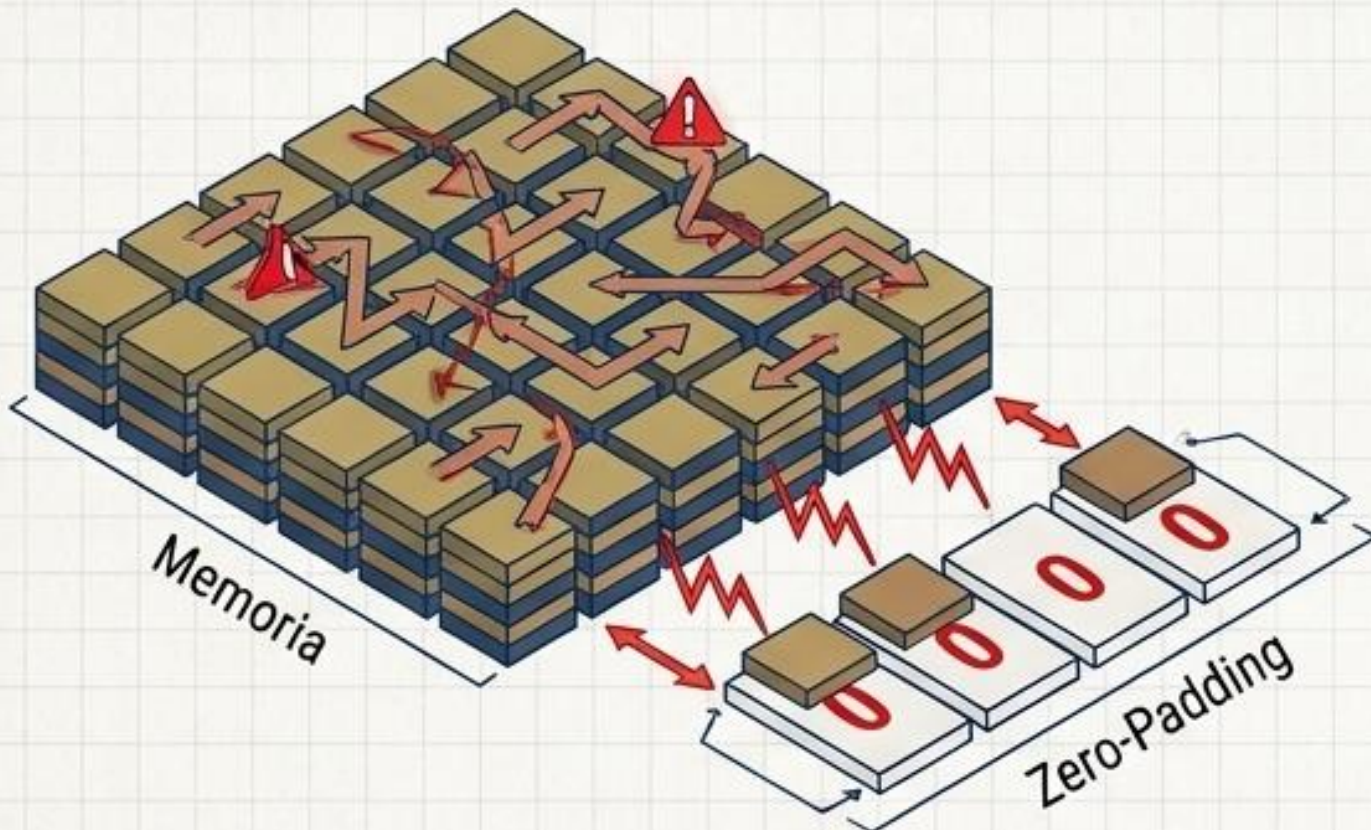
¿Cómo multiplicamos matrices densas de números de 32 bits usando instrucciones que físicamente solo aceptan vectores de 16 bits sin destruir el rendimiento?

Optimización algorítmica: Producto Interno vs. Producto Externo

Producto Interno (Inner / Dot Product)

El método escolar. Multiplica filas por columnas.

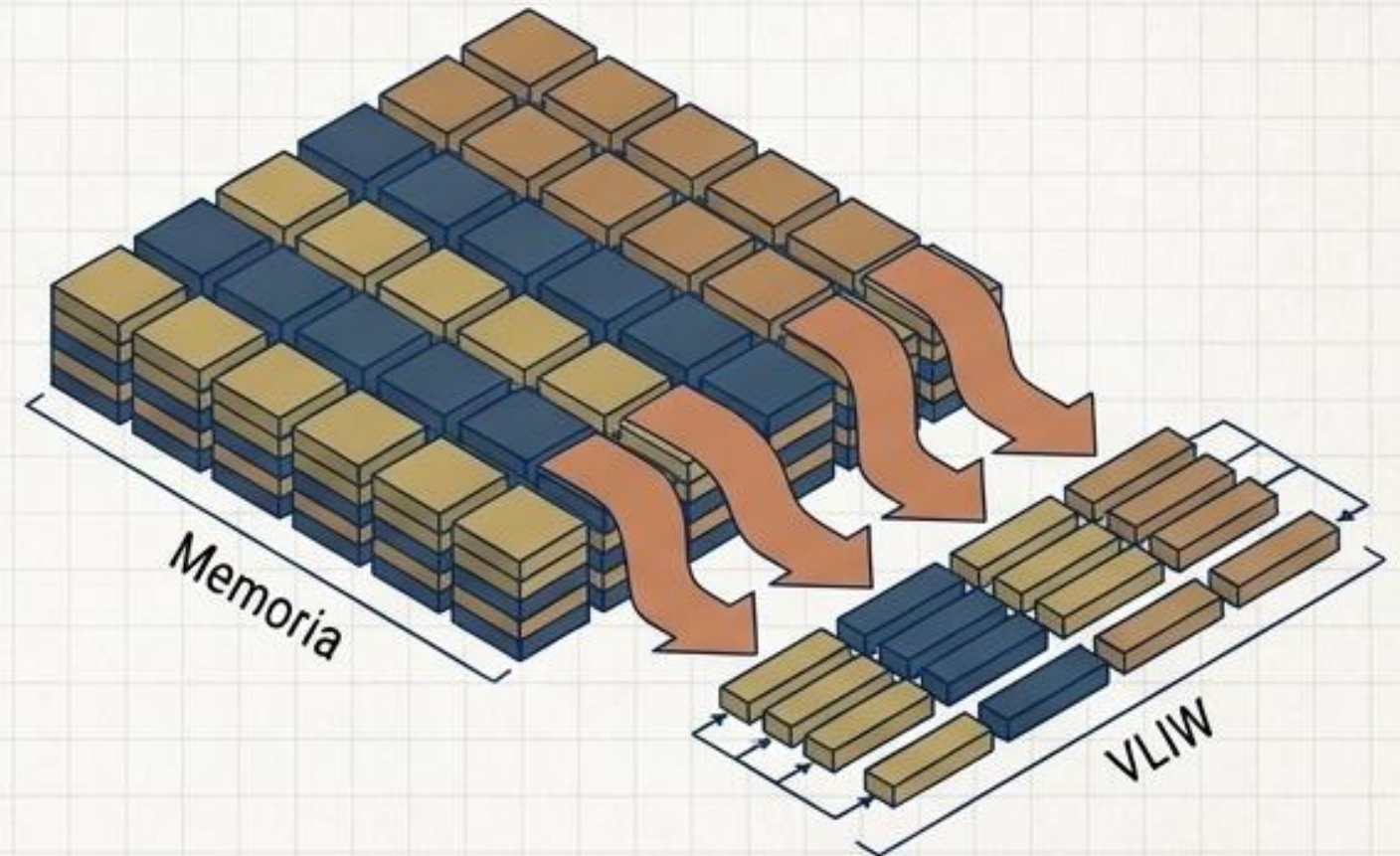
Terrible para SIMD. Las columnas no residen de forma contigua en memoria. Requiere transposición costosa o zero-padding que desperdicia ciclos de reloj.



Producto Externo (Outer / Tensor Product)

La solución SIMD. Suma de productos externos de columnas del operando izquierdo y filas del derecho.

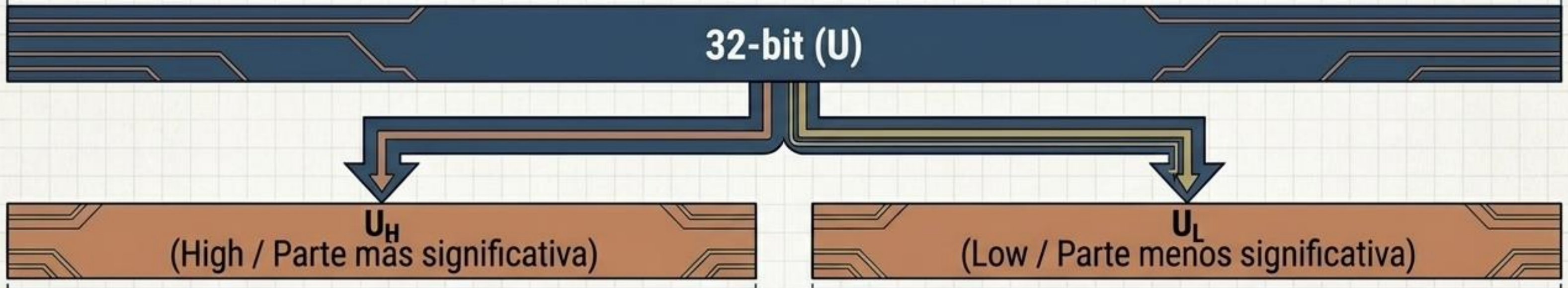
Acceso secuencial perfecto. No requiere rellenado de ceros. Utiliza toda la capacidad del pipeline VLIW simultáneamente.



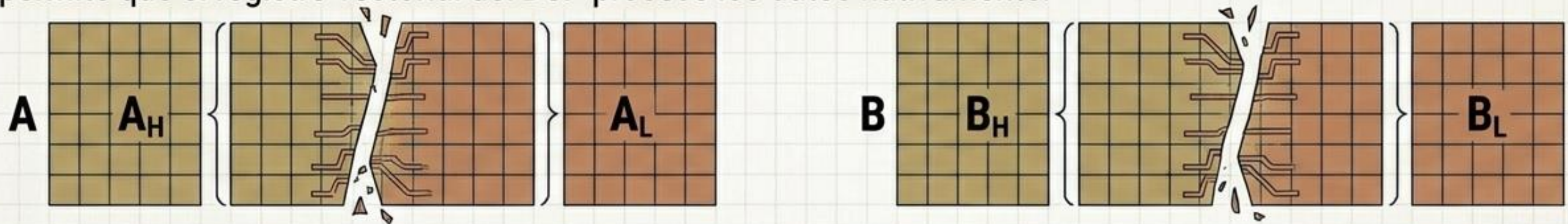
Descomposición matemática de 32 a 16 bits

Utilizando el algoritmo de Karatsuba, un entero sin signo U de n bits se divide lógicamente:

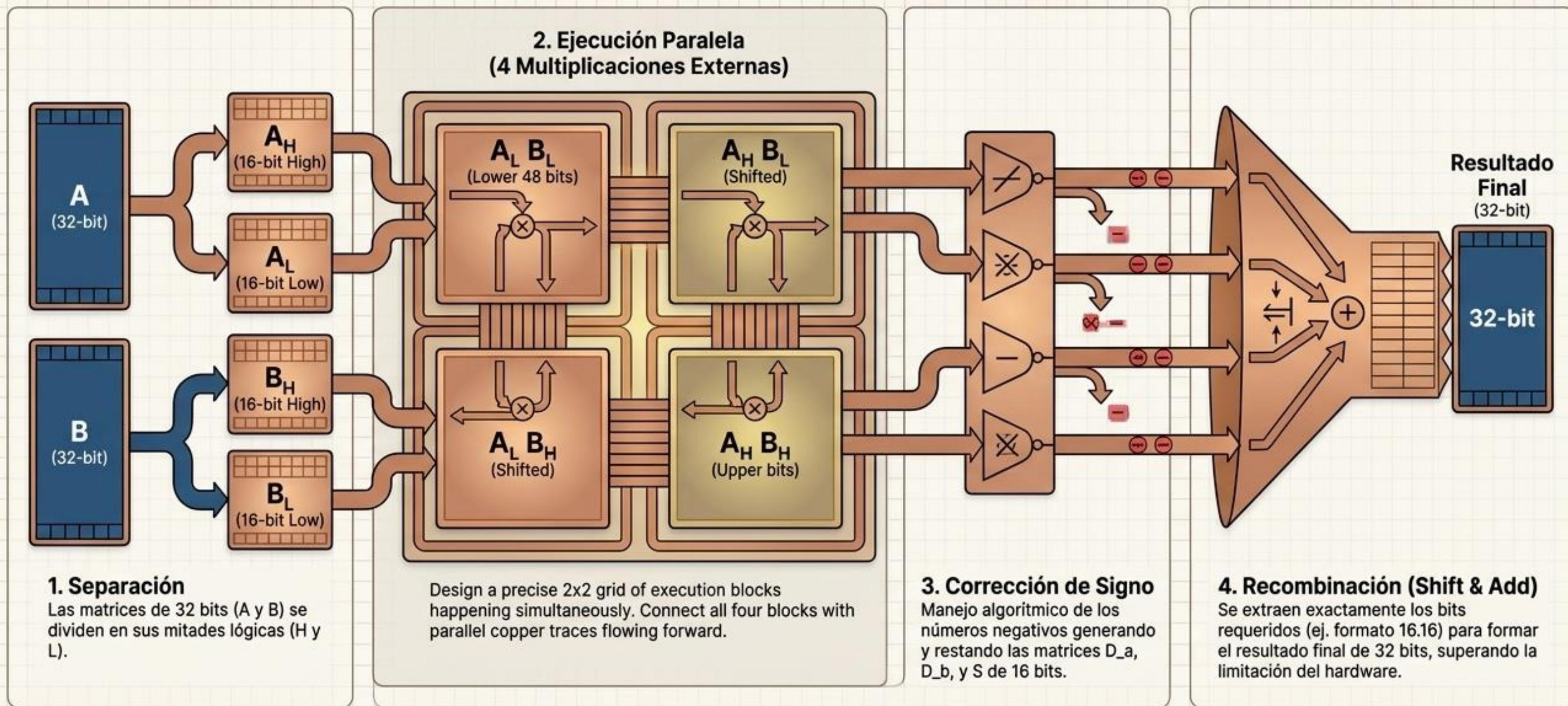
$$U = 2^{n/2}U_H + U_L$$



Para multiplicar la matriz A y B , se descomponen completamente en matrices de 16 bits: A_H , A_L , B_H y B_L . Esto permite que el registro vectorial del DSP procese los datos nativamente.



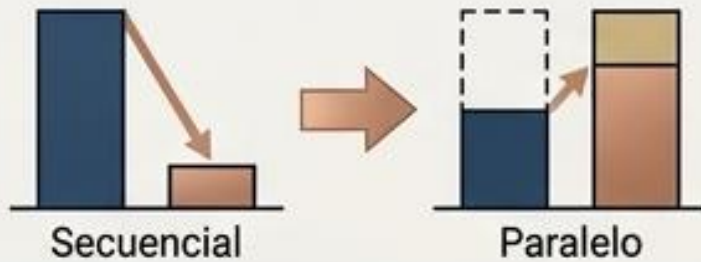
Ejecución paralela a nivel de registros



El Paradigma de Síntesis: Rompiendo el muro

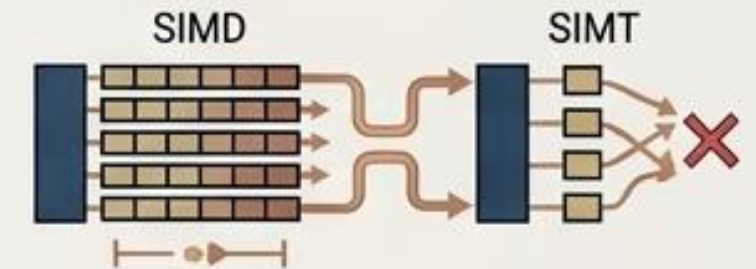
Teoría Algorítmica

Respetar las leyes de Amdahl y Gustafson minimizando las fracciones secuenciales.



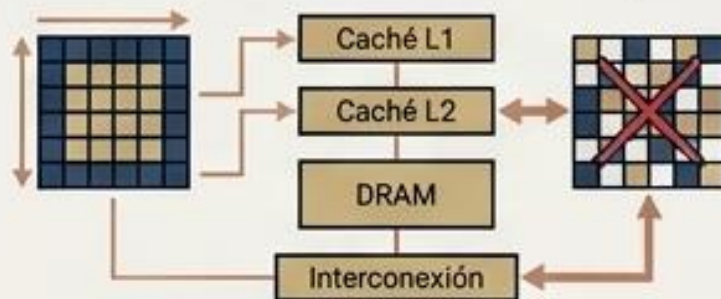
Arquitectura de Ejecución

Diseñar en torno a las limitaciones del silicio: evitar divergencias en SIMT y alinear los datos en SIMD.



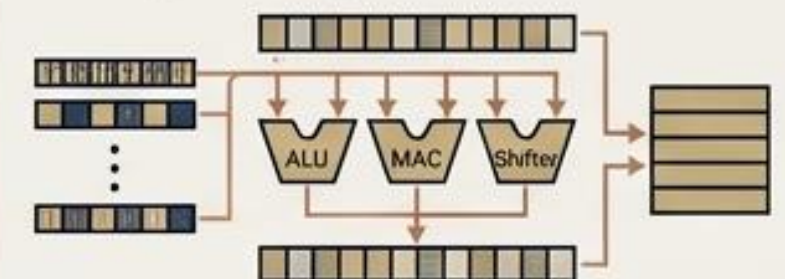
Gestión de Memoria

Organizar los datos físicamente (ej. productos externos) para evitar latencias de caché y cuellos de botella de red (MIMD).



Matemática a Bajo Nivel

Adaptar la lógica binaria y los desplazamientos a los registros físicos reales (DSP VLIW).



La computación paralela de alto rendimiento no se trata solo de añadir más procesadores. Es la alineación perfecta de múltiples disciplinas.

El máximo rendimiento se logra cuando la algoritmia de alto nivel y el silicio operan en el mismo lenguaje físico.